

BỘ MÔN KHOA HỌC DỮ LIỆU

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

LAB 07 – ARTIFICIAL INTELLIGENCE



TS. TRẦN NGỌC VIỆT



HỌC KỲ 1 – NĂM HỌC 2024-2025



KHÓA K28 & K29

#Thực hành 01:

#Thuật toán di truyền

+Thuật toán di truyền - GA là thuật toán tìm kiếm dựa trên các khái niệm về chọn lọc tự nhiên và di truyền. GA là một tập con của một nhánh tính toán lớn hơn nhiều, được gọi là tính toán tiến hóa.

+Giả sử, một nhóm các giải pháp khả thi cho vấn đề đã cho. Quá trình tái tổ hợp và đột biến (di truyền tự nhiên), tạo ra những đứa trẻ mới và quá trình này được lặp lại trong nhiều thế hệ khác nhau. Mỗi cá thể (hoặc ứng cử viên) được chỉ định một giá trị phù hợp (hàm mục tiêu) và các cá thể lai tạo có cơ hội cao hơn (trội hơn) để giao phối và sinh ra các cá thể phù hợp (học thuyết Darwin) hoặc tạo ra những người khỏe mạnh tốt nhất.

+Tiếp tục phát triển các cá nhân hoặc giải pháp tốt hơn qua nhiều thế hệ, cho đến khi đạt được tiêu chí thoả điều kiện thì dừng.

#Thực hành 01:

#Các bước thuật toán:

Các bước của cơ chế thuật toán Di truyền - GA được sử dụng để tối ưu hóa các vấn đề

B.1: Tạo ngẫu nhiên quần thể ban đầu.

B.2: Chọn giải pháp ban đầu với các giá trị phù hợp nhất.

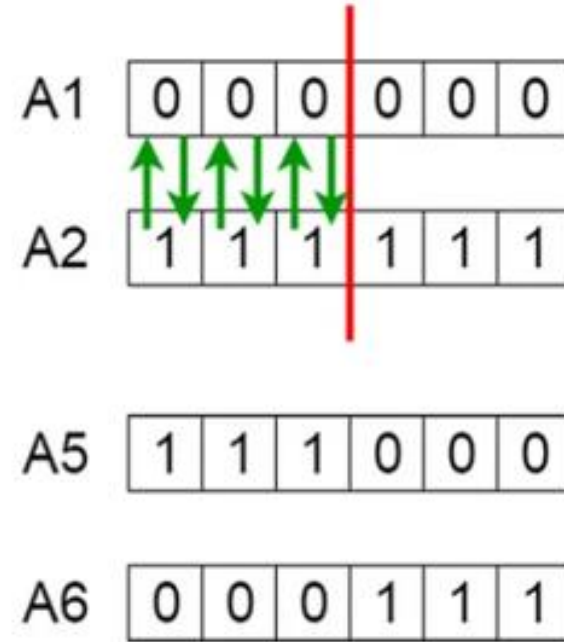
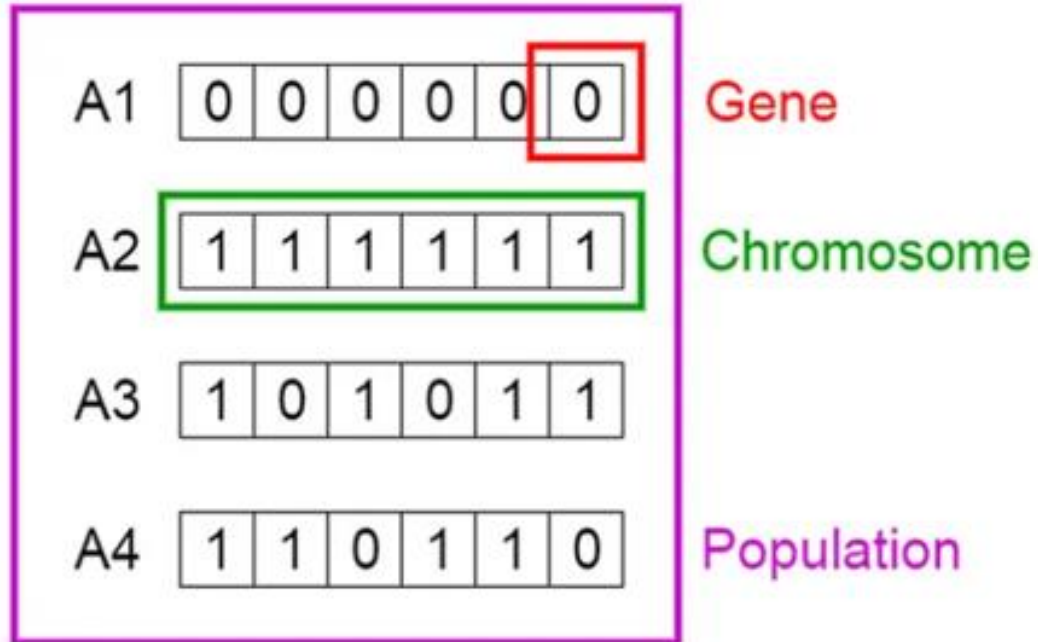
B.3: Tổng hợp lại các giải pháp đã chọn bằng cách sử dụng các toán tử đột biến và chéo.

B.4: Đưa một con lai vào quần thể.

B.5: Nếu thoả điều kiện dừng được đáp ứng, trả về giải pháp với giá trị thể lực tốt nhất của quần thể. Sau đó, chuyển sang lặp Bước 2.

#Thực hành 01:

Genetic Algorithms



#Thực hành 01:

#Genetic Algorithms – thuật toán di truyền

```
import numpy as np
#định nghĩa hàm fitness_function - hàm số âm
def fitness_func(individual):
    return -np.sum(individual**2)

#tạo quần thể ban đầu
def generate_population(size, dim):
    return np.random.rand(size, dim)

#định nghĩa hàm Genetic algorithm
def genetic_algorithm(population, fitness_func, n_generations=100,
mutation_rate=0.01):
    for _ in range(n_generations):
        population = sorted(population, key=fitness_func, reverse=True)
        next_generation = population[:len(population)//2].copy()
        while len(next_generation) < len(population):
            parents_indices = np.random.choice(len(next_generation), 2,
replace=False)
            parent1, parent2 = next_generation[parents_indices[0]],
next_generation[parents_indices[1]]
            crossover_point = np.random.randint(1, len(parent1))
            child = np.concatenate((parent1[:crossover_point],
parent2[crossover_point:]))
            if np.random.rand() < mutation_rate:
                mutate_point = np.random.randint(len(child))
                child[mutate_point] = np.random.rand()
            next_generation.append(child)
        population = np.array(next_generation)
    return population[0]
```

#Thực hành 01:

```
#tham số
population_size = 10
dimension = 5
n_generations = 50
mutation_rate = 0.05
#khởi tạo quần thể
population = generate_population(population_size, dimension)
#run - thuật toán di truyền
best_individual = genetic_algorithm(population, fitness_func,
n_generations, mutation_rate)
print("Best individual:", best_individual)
print("Best fitness:", -fitness_func(best_individual))
#chuyển đổi sang giá trị dương
```

#out:

```
Best individual: [0.04874523 0.164822    0.0404535
0.1900449 0.167309  ]
Best fitness: 0.09528824076783039
```

#nhận xét: input, output?

#Thực hành 02:

#thuật toán tiến hóa vi phân đột biến-Differential Evolution (DE)

- Thuật toán tiến hóa khác là thuật toán tiến hóa vi phân nhằm tìm kiếm sự tối ưu của một vấn đề nào đó bằng cách sử dụng các cải tiến cho một giải pháp ứng cử viên.
- Tiến hoá vi phân hoạt động bằng cách đưa ra các giải pháp ứng cử viên mới từ quần thể thông qua một hoạt động được gọi là phép cộng vector (phép đột biến và lai ghép).
- DE thường được thực hiện bằng các hoạt động đột biến và lai ghép để tạo ra các vector mới và thay thế các cá thể có mức độ phù hợp thấp trong quần thể.

#Thực hành 02:

```
import numpy as np
def differential_evolution(objective_func, bounds, pop_size=50,
max_generations=100, F=0.5, CR=0.7, seed=None):
    np.random.seed(seed)
    n_params = len(bounds)
    population = np.random.uniform(bounds[:, 0], bounds[:, 1],
size=(pop_size, n_params))
    best_solution = None
    best_fitness = np.inf
    for generation in range(max_generations):
        for i in range(pop_size):
            target_vector = population[i]
            indices = [idx for idx in range(pop_size) if idx != i]
            a, b, c = population[np.random.choice(indices, 3,
replace=False)]
            mutant_vector = np.clip(a + F * (b - c), bounds[:, 0],
bounds[:, 1])
            crossover_mask = np.random.rand(n_params) < CR
            trial_vector = np.where(crossover_mask, mutant_vector,
target_vector)
            trial_fitness = objective_func(trial_vector)
            if trial_fitness < best_fitness:
                best_fitness = trial_fitness
                best_solution = trial_vector
            if trial_fitness <= objective_func(target_vector):
                population[i] = trial_vector

    return best_solution, best_fitness
```

#Thực hành 02:

```
#hàm mục tiêu (hàm min)
def sphere_function(x):
    return np.sum(x**2)
#xác định tham số
bounds = np.array([[ -5.12, 5.12]] * 10) # Example: 10 parameters in [-
5.12, 5.12] range
#run tiến hoá DE
best_solution, best_fitness = differential_evolution(sphere_function,
bounds)
#kết quả giải pháp tốt nhất & chất lượng tốt nhất
print("Best solution:", best_solution)
print("Best fitness:", best_fitness)
#out:
```

```
Best solution: [ 0.00159031 -0.00567955 -0.00020499 -0.00573414  0.00164174 -0.0051367
 0.00190499 -0.01349925 -0.00127274 -0.00299844]
Best fitness: 0.00029325913422526543
```

#nhận xét: input, output?

#Thực hành 03:

```
#Bài toán Travelling SalesMan(bằng thuật toán di truyền)
import random
import copy
import math
import matplotlib.pyplot as plt

POPULATION_SIZE = 20    #số quần thể
CITIES_SIZE = 20        #số thành phố
TOUR_SIZE = 21          #số tour du lịch
NUM_EXECUTION = 9999    #số lần thực hiện
population = []
x = []
y = []
tour = [[0 for x in range(TOUR_SIZE)] for y in range(TOUR_SIZE)]
dCidade = [[0 for x in range(POPULATION_SIZE)] for y in
range(POPULATION_SIZE)]
distances = [0 for x in range(POPULATION_SIZE)]
parentsOne = None
parentsTwo = None
costByExecution = []
```

#Thực hành 03:

```
def generateFirstPopulation():    #tạo quần thể đầu tiên
    #tạo đường đi mới
    for _ in range(1, POPULATION_SIZE + 1):
        generatePossiblePath()

def generatePossiblePath():    #tạo mới tập hợp đường đi
    path = []
    for _ in range(1, CITIES_SIZE + 1):
        #tạo mới số cá thể từ 1-20
        randomNum = random.randint(1, 20)
        while (numberExistsInPath(path, randomNum)):
            randomNum = random.randint(1, 20)
        path.append(randomNum)
    population.append(path)

def numberExistsInPath(path, number):
    for i in path:
        if i == number:
            return True
    return False
```

#Thực hành 03:

#Tạo mảng X và mảng Y - khoảng cách trong trục x và y để tính ma trận nhận dạng trong hàm thích nghi

```
def generateXandY():
```

```
    for _ in range(CITIES_SIZE):
```

```
        randomNumber = random.random()
```

```
        randomNumber = round(randomNumber, 2)
```

```
        x.append(randomNumber)
```

```
        randomNumber = random.random()
```

```
        randomNumber = round(randomNumber, 2)
```

```
        y.append(randomNumber)
```

#hoán đổi giữa 2 thành phố trên đường đi, với 5% cơ hội đột biến

```
def mutate(matrix):
```

```
    for i in range(0, len(matrix)):
```

```
        for _ in range(0, len(matrix[i])):
```

```
            ranNum = random.randint(1, 100)
```

```
            if ranNum >= 1 and ranNum <= 5:
```

```
                indexOne = random.randint(0, 19)
```

```
                indexTwo = random.randint(0, 19)
```

```
                auxOne = matrix[i][indexOne]
```

```
                auxTwo = matrix[i][indexTwo]
```

```
                matrix[i][indexOne] = auxTwo
```

```
                matrix[i][indexTwo] = auxOne
```

#Thực hành 03:

#Tạo ma trận Tour, ma trận giống như quần thể, với cột đầu tiên được sao chép ở cuối cột đó, sau cùng, người du lịch luôn phải đến cùng một nơi mà anh ta đã xuất phát

```
def generateTour():  
    global tour  
    tour = copy.deepcopy(population)  
    for ways in tour:  
        first = ways[0]  
        ways.append(first)
```

#Tạo mảng có tổng đường đi trong mảng quần thể dựa trên ma trận tour;

```
def calculateDistances():  
    global distances  
    distances = [0 for x in range(POPULATION_SIZE)]  
    for i in range(len(population)):  
        for j in range(len(population[i])):  
            firstPos = 19 if tour[i][j] == 20 else tour[i][j]  
            secondPos = 19 if tour[i][j+1] == 20 else tour[i][j+1]  
            distances[i] += round(dCidade[firstPos][secondPos], 4)  
    dict_dist = {i: distances[i] for i in range(0, len(distances))}  
    distances = copy.deepcopy(dict_dist)  
    return sorted(distances.items(), key=lambda kv: kv[1])
```

#Thực hành 03:

#Tạo ma trận nhận dạng (dCidade) dựa trên mảng x và y, rồi gọi hàm CalculateDistances() để tạo mảng có tổng đường đi tới người dùng sau, theo chu trình;

```
def fitnessFunction():  
    for i in range(len(population)):  
        for j in range(len(population)):  
            dCidade[i][j] = round(math.sqrt(((x[i] - x[j])**2) + ((y[i] - y[j])**2))), 4)  
    return calculateDistances()
```

#hàm roulette, tạo ra hai mảng, mỗi mảng sẽ được sử dụng sau, thực hiện theo chu trình;

```
def rouletteFunction(sorted_x):  
    global parentsOne  
    global parentsTwo  
    arr = []  
    rouletteArr = []  
    for i in range(10):  
        arr.append(sorted_x[i][0])  
    for j in range(len(arr)):  
        for _ in range(10 - j):  
            rouletteArr.append(arr[j])  
    parentsOne = createParents(rouletteArr)  
    parentsTwo = createParents(rouletteArr)
```

#Thực hành 03:

```
#sử dụng hàm rouletteFunction() để tạo ra hai mảng cha;
def createParents(rouletteArr):
    parentArr = []
    for _ in range(5):
        parentArr.append(rouletteArr[random.randint(0, 54)])
    return parentArr

#sử dụng phương pháp tuần hoàn để xem xét thành phố nào bị trùng lặp
hay không
def hasDuplicity(auxArray, usedIndexes):
    for i in range(len(auxArray)):
        for j in range(i, len(auxArray)):
            if i != j and auxArray[i] == auxArray[j]:
                if i in usedIndexes:
                    return j
                else:
                    return i
    return -1
```

#Thực hành 03:

#1. hai phần tử con trong mảng con, thực hiện hoán đổi ngẫu nhiên cho đến khi không còn phần tử trùng lặp

#2. Thay đổi children được tạo ra

#3. Thêm các phần tử children vào mảng quần thể

```
def doCycle(sorted_x):  
    global population  
    children = []  
    for i in range(5):  
        parentOneAux = parentsOne[i]  
        parentTwoAux = parentsTwo[i]  
        usedIndexes = []  
        randomIndexInsideCromossomus = random.randint(0, POPULATION_SIZE -  
1)  
  
        usedIndexes.append(randomIndexInsideCromossomus)  
        childOne = copy.deepcopy(population[parentOneAux])  
        childTwo = copy.deepcopy(population[parentTwoAux])  
  
        valAuxOne = childOne[randomIndexInsideCromossomus]  
        valAuxTwo = childTwo[randomIndexInsideCromossomus]
```

#Thực hành 03:

```
childOne[randomIndexInsideCromossomus] = valAuxTwo
childTwo[randomIndexInsideCromossomus] = valAuxOne

while (hasDuplicity(childOne, usedIndexes) != -1):
    newIndex = hasDuplicity(childOne, usedIndexes)
    usedIndexes.append(newIndex)
    valAuxOne = childOne[newIndex]
    valAuxTwo = childTwo[newIndex]
    childOne[newIndex] = valAuxTwo
    childTwo[newIndex] = valAuxOne

# Sau khi tạo ra phần tử con, thêm chúng vào mảng phần tử con;
children.append(childOne)
children.append(childTwo)

#thay đổi mảng con
mutate(children)

#Tạo bản sao quần thể trước khi thay đổi nó
tempPop = copy.deepcopy(population)

for i in range(10):
    population[i] = copy.deepcopy(tempPop[sorted_x[i][0]])

#điều chỉnh quần thể
for j in range(10, POPULATION_SIZE):
    population[j] = copy.deepcopy(children[j - 10])
```

#Thực hành 03:

```
def main():
    #tạo quần thể, x và y và ma trận tour;
    generateFirstPopulation()
    generateXandY()
    generateTour()

    #vòng lặp từ 0 - 9999 ;
    for _ in range(NUM_EXECUTION):
        sorted_x = fitnessFunction()
        rouletteFunction(sorted_x)
        doCycle(sorted_x)
        generateTour() #lặp ma trận Tour khi quần thể được cập nhật
        costByExecution.append(sorted_x[0][1])
        #Thêm chi phí vào mảng chi phí

    #Tạo giá trị thích hợp cho quần thể cuối cùng
    sorted_x = fitnessFunction()

    print('Total Population: %s' % (POPULATION_SIZE))
    print('Mutation Probability: 5%') #xác suất đột biến
    print('No of Cities: %s' % (CITIES_SIZE))
    print('optimal Path Cost: %s' % sorted_x[0][1])
    print('Best Route: %s' % population[0]) #tuyến đường đi tốt nhất
```

#Thực hành 03:

```
#biểu đồ đường đi
plt.plot(tour[0])
plt.plot(tour[0], 'rs')
plt.axis([0, 20, 0, 20])
plt.xlabel('No of cities') #số thành phố (số điểm)
plt.ylabel("Total population") #tổng số quần thể
plt.title("The route of Salesman")
plt.show()

#biểu đồ chi phí
plt.plot(costByExecution)
plt.xlabel('No of Iterations') #số lần lặp
plt.ylabel("Cost function value (fitness value)") #hàm chi phí
plt.show()

if __name__ == "__main__":
    main()
```

#Thực hành 03:

```
#biểu đồ đường đi
plt.plot(tour[0])
plt.plot(tour[0], 'rs')
plt.axis([0, 20, 0, 20])
plt.xlabel('No of cities') #số thành phố (số điểm)
plt.ylabel("Total population") #tổng số quần thể
plt.title("The route of Salesman")
plt.show()

#biểu đồ chi phí
plt.plot(costByExecution)
plt.xlabel('No of Iterations') #số lần lặp
plt.ylabel("Cost function value (fitness value)") #hàm chi phí
plt.show()

if __name__ == "__main__":
    main()
```

#Thực hành 03:

#out:

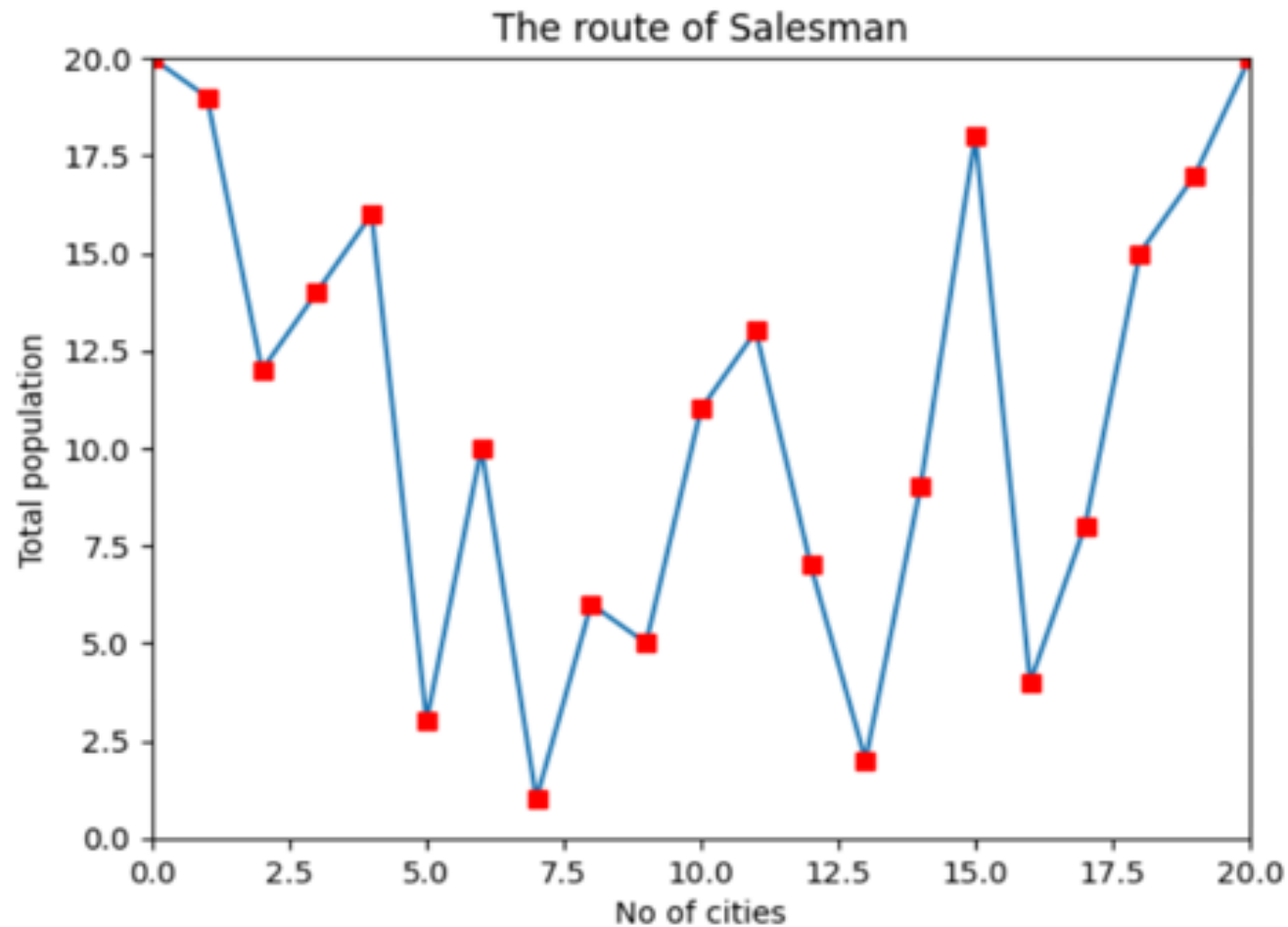
Total Population: 20

Mutation Probability: 5%

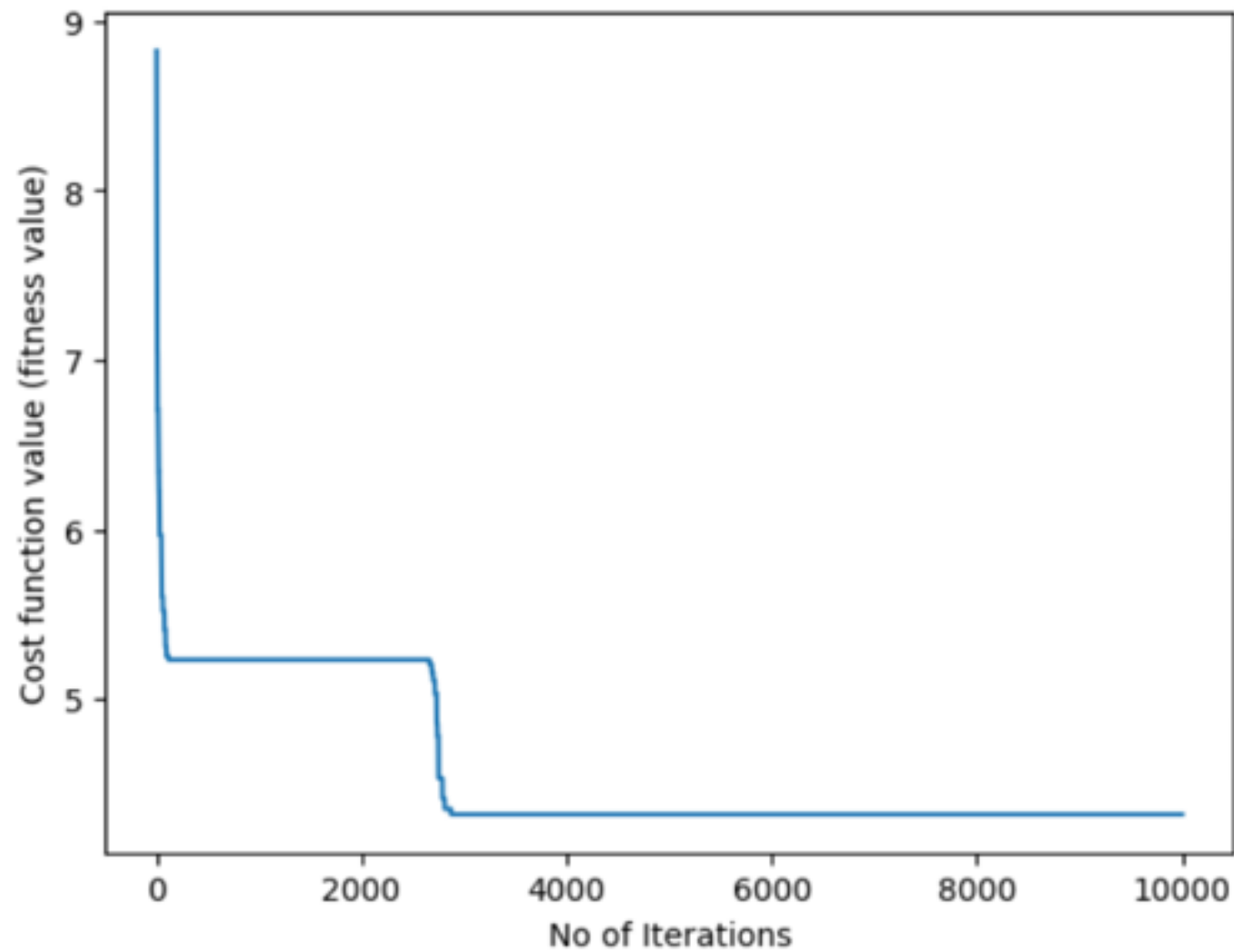
No of Cities: 20

optimal Path Cost: 4.316999999999999

Best Route: [20, 19, 12, 14, 16, 3, 10, 1, 6, 5, 11, 13, 7, 2, 9, 18, 4, 8, 15, 17]



#Thực hành 03:



#nhận xét: input, output?



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

